Lecture 4 Memory Management

2022年12月14日 8:58

Introduction

We will focus on Intel's memory management (x86) when we talk about practical things
- Process level (logical level)
- Kernel level (mapping from logical level to physical level)
- Device level (physical level)

MM from a process' perspective

- Local variables: stack
- Dynamically allocated memory: heap
- Global variables: data segment
- Program code and constants: code/text segment

- Constants: fixed total space, and read-only.
- Global variables: fixed total space, but not read-only.
- Local variables: unfixed total space, and not read-only.
- Allocated memory: unfixed total space, and not read-only.
- Program code: unfixed total space, and not read-only (there is something called the self-modifying code).

Program code

A process is running on some **program code**. The process can change this relationship using the *exec*()* family of system calls. The running program code is required to be **in the memory**, since the CPU needs to fetch the instructions from the memory for execution.

Global variables and constants

Global variables and constants are **fixed at compile time**. When a new program code starts running (started by an *exec*()* system call), the global variables and the constants are created in the memory.

Global variables and constants are actually managed differently. We can modify the values of the global variables (size unchanged), but constants are **read-only** and should be protected. For instance, assigning a char pointer to "Hello world" and trying to modify the pointer[0] will give a **segmentation fault**.

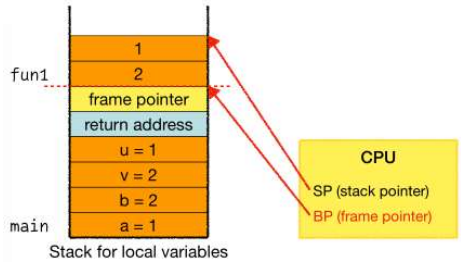
Local variables

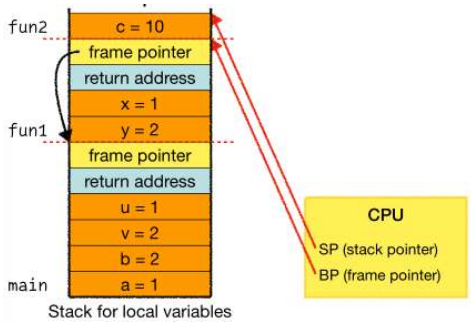
Local variables are **bound to a function only**. When a function is invoked, the local variables are created, and when the functions returns, those local variables are abandoned. Note that <u>the kernel is not involved during the function calls, and the **compiled code** contains all the memory management</u>.

- Local variables are organized as a **stack**. When the *main()* function is invoked, its local variables are pushed onto the stack.
- Function parameter passing is also done through the stack: before the program counter jumps to some function, its **parameters** and the **return address** are pushed onto the stack*. The function refers to the parameters as if they are local variables since they are all in the stack. It also pushes the previous **frame pointer**. If there are new local variables in the called function, these new local variables will also be pushed onto the stack.
- When a function returns, all the local variables and functions parameters are **popped out** (logically, meaning that they are not zeroed out in the memory).
- The return value is stored in the **register EAX****, and the value of EAX will be copied to the return address upon return.
- When the main function finishes, the *exit()* system call is invoked, and all the process' memory is destroyed.

* On x86_64, the first six parameters are passed using registers, and the remaining parameters are pushed onto the stack.
** On x86_64, the return value is stored in the register RAX.

The implementation of the stack
- The CPU has a register called the **stack pointer (SP)**, which always points to the **top** of the stack.
- The stack pointer moves up and down to define the stack space. A function can ask the CPU to read/write anywhere in the stack, not just the top of the stack and not necessarily within the stack frame of the current function.
- The **frame pointer** (a.k.a **based pointer (BP)**) register points to the stack top <u>as it was when the function was just invoked</u>.





- The total size of the local variables changes **dynamically**. The compiler cannot estimate the size of the stack at compile time, since the number of function calls depends on many factors. What the kernel can do is just to reserve a large enough space for the stack.
- If there is no more space left in the stack, this will lead to a **stack overflow**.
- If a process needs more memory space, it will use the dynamically allocated memory, as we will discuss next.

Dynamically allocated memory

Dynamically allocated memory is **bound to the entire process**. The space is often called the **heap** (not organized as a binary heap structure). Once allocated, the memory can be accessed by **every** function of the program. Unlike local variables, the memory allocation is implemented by the **kernel**.

The kernel controls **all** the memory in the system. When you call *malloc()*, the kernel tries to find some memory for you.

- Note that *malloc()* is not a system call, but a function in the C standard library.
- Allocating memory is done by the *brk()* or *mmap()* system call.
  - The *brk()* system call can grow or shrink the allocated area.
  - The *mmap()* system call can map files or devices into memory.
- *malloc()* only manages the free space returned by the kernel.

When you ask *malloc()* for some bytes, it actually asks for more bytes using the *brk()* or *mmap()* system call. Note that it internally needs memory to manage the returned free space (thus some **overhead** for each call). For instance, *malloc*ing for each byte of a large file versus *malloc*ing the whole file once, the first version may run out of memory since there is some overhead for each call.

Segmentation

Segmentation is to view the memory of a process as a set of segments instead of an array of bytes. The access to different segments is controlled by the CPU. Note that if you access a piece of memory that is not allowed to be accessed, the OS will raise a **segmentation fault**! For instance,

- Modifying read-only segments
- Accessing unallocated segments
- Stack overflow (because you are trying to access unallocated heap space)
- Dereferencing a *NULL* pointer (because you are not allowed to visit the memory address 0x00000000)
- etc.

MM from the kernel's perspective

The kernel knows how many processes there are in the system, how much space each process needs, and how much memory there is in the system. But what if the space needed by all processes exceeds the total memory in the system?

Physical memory layout

Recall that the main memory is divided into the kernel space memory (the running kernel and its data) and the user space memory (the processes).

If there are too many processes in the system that causes a main memory overflow, we can use hard disk's space to store the processes. The portion of the hard disk will be reserved to serve as the memory, which is called the **swap**.

Swapping

When a process is scheduled, it needs memory space to run. But the scenario is, the physical memory is fully occupied. Then, one of the idle (not running) processes will be **swapped out** of the main memory, and the swapped-out process will be stored inside the **swap**. That process will be **swapped in** from the disk if it is scheduled again.

Other than supporting more processes in the system, swapping is actually a bad thing: it increases the time in context switching, and it causes the fragmentation problem (in particular, **external fragmentation**, where there are holes but not big enough).

We could use **defragmentation** (a.k.a. **memory compaction**) to pack the processes. However, the memory compaction process takes a lot of time, since the CPU needs to move the memory in a unit-by-unit manner. This further delays context switching.

Address space

When the CPU wants to read/write a piece of memory, a **memory address** is needed. In x86 (32-bit), addresses are 32-bit wide, so the addressable memory space is 2^32=4GB*.

*In x86_64 (64-bit), addresses are 64-bit wide (but only the lower 48 bits are used currently), so the addressable memory space is 2^48=256TB.

In the swapping example, the physical memory location of Process A may be changed when it is rescheduled, which implies that the same variable may have **different addresses** at different times. After a program has been compiled, the variables names are translated into memory addresses. However, due to swapping the physical addresses of the variables may be changed, and there is no way for a program to run in this case.

The fact is: **the compiled code is not using physical addresses, but using logical addressing**. A logical address can be translated to a physical address by the OS kernel or the CPU (there is a chip called the **memory management unit (MMU)** inside the CPU).

A **logical address** is an **offset** inside the memory of a process. Every process has a **base address**, which is a physical address, as well as the lowest address value of the process (the base address may be changed after swapping since it is physical). The kernel maintains **a table of base addresses**. With this approach, the compiled program only needs to maintain the offsets (*i.e.*, the logical addresses) instead of hardcoding the physical addresses, which solves the problem caused by swapping.

What if … a process' allocated memory grow? And recall that we also want to handle external fragmentation problem without memory compaction since it is time-inefficient.

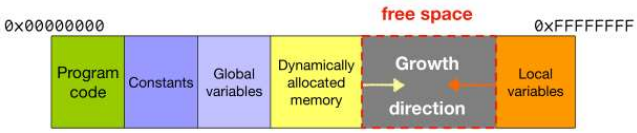
Virtual memory

Virtual memory is a memory management technique that solves all the problems previously mentioned:
- External fragmentation problem
- Memory growth problem
- Memory size limit problem

Virtual memory allows:
- The physical address space of processes can be **noncontiguous**.
- Processes can run even when they are only **partially** in the memory.

Virtual memory layout from the process' perspective

Every process has almost the **entire** address space. The heap and the stack grow inside the process' memory, one from the beginning and one from the end, growing in opposite directions.

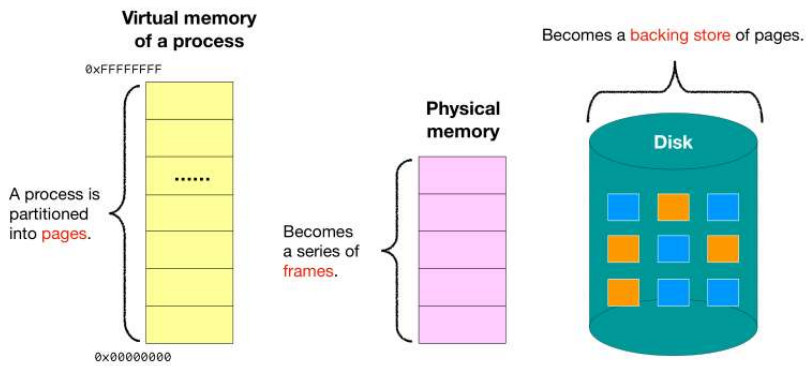


Virtual memory layout from the kernel's perspective

The physical memory is partitioned into fixed-size (*e.g.*, 4KB) blocks called **frames** (a.k.a. **page frames**). A process' logical memory is also partitioned into block of the same size called **pages**. When a process requires memory, the memory is allocated in terms of pages. Note that this may lead to internal fragmentation.

When a process is loaded into the memory, the memory is anchored to the frames. In order to support a process memory larger than the physical memory size, it is **not necessary** that all the pages of a process are loaded into the physical memory all at once. The missing pages are on the disk, which will be loaded into the frame on demand. Therefore, the virtual memory allows a process' memory to **grow**: the kernel does not allocate **any** page to the free space until *brk()* or *mmap()* is invoked. That is to say, *malloc()* is actually finding you **free pages** for the "dynamically allocated memory" to grow.

Virtual memory internals



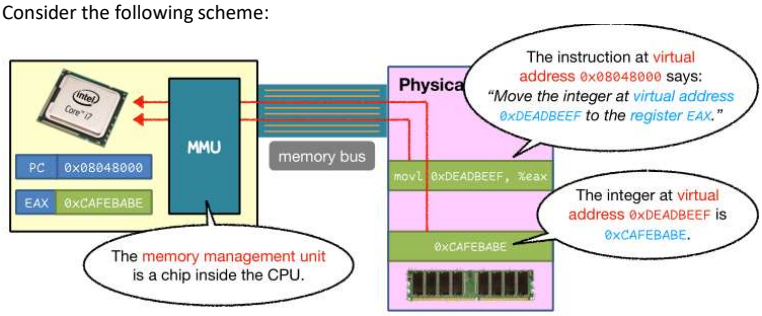
Some of the yellow blocks are pointing to the pink blocks, and some are pointing to the orange blocks. Therefore, the memory of a process (yellow) can **grow** beyond the size of the physical memory (pink).

The CPU runs in a **fetch-decode-execute cycle**.
1. Fetch instruction from the memory.
2. Decode it.
3. Execute it.

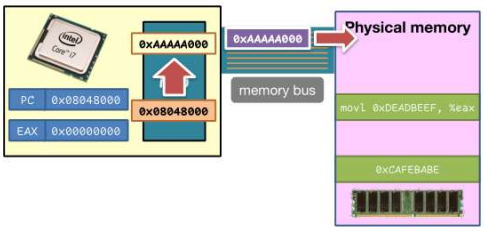
We need to translate virtual addresses into physical addresses, since CPU only works closely with the hardware but it does not know how to read virtual addresses.

Virtual memory support in modern CPUs

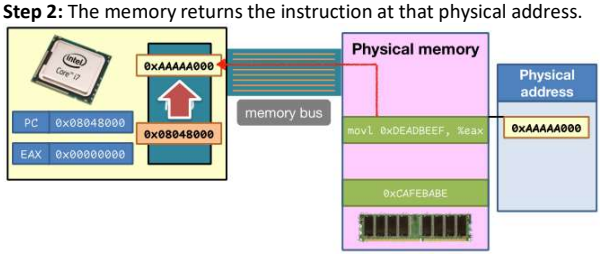
Consider the following scheme:



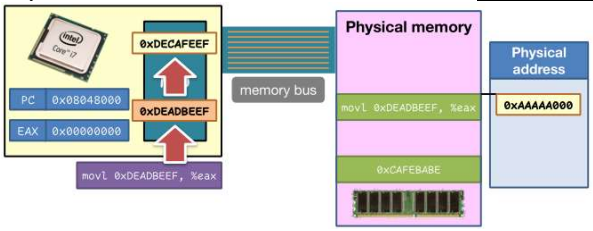
- **Step 1:** To fetch an instruction, the CPU sends the virtual address to the MMU, and the MMU translates it into the physical address



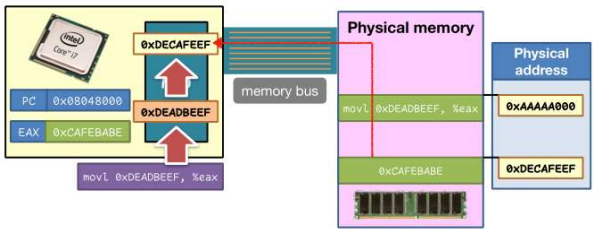
- **Step 2:** The memory returns the instruction at that physical address.



- **Step 3:** The CPU decodes the instruction. An instruction <u>always uses virtual addresses</u>, not physical addresses!

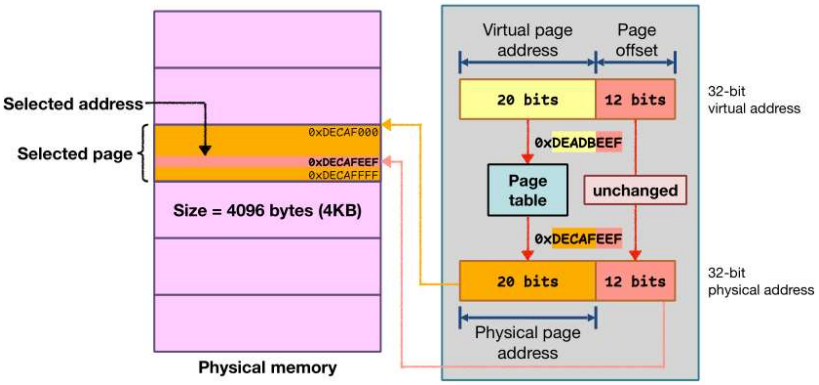


- **Step 4:** With the help of the MMU again, the target memory is retrieved.



Page table

A **page table** stores the **memory mapping**. It tells which pages are in the physical memory, and is stored in the memory and used by the MMU. The content of the page table depends on the paging mechanism.



Note that the 32-bit virtual address is split into the 20-bit virtual page address and the 12-bit page offset (page size is 4KB, so 12 bits). The page table deals only with the **high 20 bits** and translate it into the physical page address.

To summarize, the process only uses the virtual address, in which there are text, initialized data, uninitialized data, allocated heap, and allocated stack. The unallocated zone is between the allocated heap and the allocated stack, and can be occupied by both (from different directions). Allocated memory are broken into pages, while unallocated zone does not occupy any pages. Memory pages are then distributed on the physical memory or the disk.

Demand paging

Demand paging loads a page from the disk to the physical memory when that page is demanded. It is similar to swapping, but the system will not load the entire process into memory, but only the pages required. This allows even **more processes** to be hosted in the system.

The system can support more processes because only part of the process is needed to be in the memory. The kernel can allow **different page frames** to hold **different processes**. However, this may increase the number of disk accesses, since a process may need several pages during the execution.

The **page table** contains the entire view of the physical memory for each process. The column "Page #" denotes the page number, the column "Frame #" denotes the corresponding frame number, and the column "Valid bit" tells if a page is in the memory ('v' for valid and 'i' for invalid (maybe on the disk)). Each row (entry) is a page.

Issue #1

Do we need to access the memory twice for every memory access? Currently, for each access of the memory, CPU talks to the MMU, the MMU looks at the page table, the MMU gets the return of the page table, and finally the MMU fetches the instruction/data from the real page.

Solution: Translation lookaside buffer (TLB), which is an address-translation **cache** inside the MMU. It stores the **recent translations** of virtual memory to physical memory.

- When a TLB hits, MMU does not need to look at and get return from the page table, but refers to the TLB.
- When a TLB misses, we still need to use the original method. But the TLB updates itself, so next time we may have better performance.

Issue #2

Unlike virtual pages, the page table must be contiguous. What if the virtual address space is too large? Consider a system with a 32-bit logical address space: a page table consists of 2^20=1 million entries. Assuming each entry consists of 4 bytes, then each process needs 4MB of contiguous physical address space just for the page table alone!

Solution: Multilevel page tables, which is a page table of page tables. With that, we do not need to keep all page table entries in memory at all times. For instance, instead of having a 20-bit physical page address, we can split it into a 10-bit outer page address and a 10-bit inner page address. If an outer page table entry is allocated, then it points to an inner page table (otherwise we do not even need to create an inner page table).

Back to our story

Our scheme is: some time in a process, the CPU does not know where to fetch the instruction because the instruction is not in any frame. But the page table know about this, since it will show the **valid bit** as **i** (invalid). Then, the kernel steps in, locating the required memory page on the backing storage (*e.g.*, disk). Then, the kernel copies the memory page into a free frame, and update the page table (and **i** to **v**). After all these, the kernel will let the interrupted process resume execution, and this time the instruction can be fetched correctly.

The sum up our previous argument, the swapping decision and operation are handled as if an **exception** has happened. When a page is not found in the main memory, an exception called **page fault** is caught. The **page fault handler** inside the kernel then

- Locates the memory page on the disk
- Chooses the right frame to store the required page in the physical memory
- Updates the page table
- Resumes the program execution

Pure demand paging

The rule is: never bring a page into memory until it is required. Therefore, when a new process is forked, none of the pages of the new process is in the memory. Then, when the child process is executed, a page fault must be raised.

Note that however, this is an extreme solution. The new process produced by the *fork()* system call has the same code/text, stack, and heap as its parent process. Also, it is usually followed by an *exec()* system call right away. While talking about process management, we know that it is stupid to first copy all the memory pages of the parent for the new process (*fork()*) and then throw away all the memory pages of the new process (*exec()*). There is a better solution which we will discuss next.

Copy-on-write

The **copy-on-write** technique allows the parent and the child processes to **share pages** (which means that different virtual memory addresses of different processes may point to the same physical address) after the *fork()* system call. A separate page will be copied and modified only when one of the process wants to write to a shared page.

Page replacement

We have been assuming that there are free frames in the physical memory, but what if all the frames are already occupied? We may need page replacement.

Page replacement happens when both the following two conditions hold:

- The process requests a page that **does not exist** in the physical memory
  - For instance, the CPU is fetching an instruction, and the next instruction may be on the disk.
  - For instance, an instruction accesses a piece of memory. The instruction loads a piece of memory to the register, but the target page may be on the disk.
- There are **no free frames** available in the physical memory
  - The frames can be easily filled up by launching more and more processes, or by allocating more and more memory (note we have to access the page after allocation since otherwise it will not be loaded into the physical memory because it is swapped on demand).

Step 1: decide the victim page in the physical memory. The kernel maintains some data in order to make this decision.
Step 2: swap out the victim and write it back to the disk. The page in the physical memory **overwrites** the page in the swap.
Step 3: swap in the desired frame. The desired page on the disk is written into the victim page.

Page replacement algorithm

- **First-in first-out (FIFO) page replacement:** the oldest page will be swapped out of the frames. The age of a page is counted by the time it is stored in the memory.
  - Too many page faults invoked (whenever the desired page is not in the frame).
- **Optimal page replacement:** replace the page that will not be used for the longest period of time.
  - However, we do not know about the future.
- **LRU page replacement:** every page is assigned an age. Whenever a page is used, its age is reset to 0 and the ages of all other pages are incremented by 1. Replace the page that was the least recently used (*i.e.*, the oldest frame by definition).

Performance issues

Increasing the number of page frames (so increasing the amount of physical memory) may intuitively help increase the performance, but the reality is: **not always**. This is called **Belady's anomaly**, where the number of page frames but the number of page faults also increase, leading to poorer performance.

This is due to the **principle of locality**, which means that a program tends to access the same set of memory locations **repetitively** over a short period of time (*e.g.*, loops). If there are more processes, then more pages will be allocated, then there will be fewer or no free frames, then the number of page faults will increase, and the principle of locality **further** increases the number of page faults.

Therefore, a lot of CPU cycles are wasted in replacing the pages but not doing useful jobs. The more processes in the system, the more likely this will happen. This is called **thrashing** 颠簸.